

TERRAFORM - Intro

What is the problem?

- Automation for Infrastructure.
- In Particular Automation for Cloud Infra (AWS, Azure, GCP)
- Every Cloud has their own Console portal to create resources.
- How to handle infra build on multiple cloud.
- Note:-- WE ARE NOT TALKING ABOUT APPLICATION INSTALL AUTOMATION
- Infra means.
 - Networks / Subnetworks
 - Firewalls
 - LoadBalancers
 - Storage
 - Public IPs
 - DNS Entries and much more

Solution:

Use TERRAFORM!!!!!!!!!!

- WHY NOT EXSISTING CONFIGURATION MGMT TOOL LIKE “Puppet, Chef or Ansible”
- These tools can do automation on application and services running inside the VM's.
- **TERRAFORM is a TRUE Infrastructure as a Code.**

IAC Tools -- Comparison

Feature / Aspect	Terraform	AWS CloudFormation	Azure ARM / Bicep	Pulumi	Ansible
Tool Category	Infrastructure as Code (IaC)	Cloud-native IaC	Cloud-native IaC	IaC (Programming-based)	Configuration Management & Automation
Primary Use Case	Provision infrastructure across clouds	Provision AWS resources only	Provision Azure resources only	Provision infrastructure using code	Configure servers, apps, OS, middleware
Multi-Cloud Support	Yes (AWS, Azure, GCP, OCI, VMware, etc.)	AWS only	Azure only	Yes	Limited (via modules & cloud CLIs)
Language Used	HCL (Declarative)	JSON / YAML	JSON / Bicep	Python, Go, TypeScript, C#	YAML
Declarative / Imperative	Declarative	Declarative	Declarative	Imperative / Declarative	Imperative
State Management	External state file (S3, Azure Blob, etc.)	Managed by AWS	Managed by Azure	Managed by Pulumi backend	No state tracking
Execution Model	Plan → Apply	Stack create/update	Deployment create/update	Preview → Update	Task execution (no plan)  Persistent

Intro to TERRAFORM

- Terraform is a tool for building, changing, and versioning infrastructure safely and efficiently.
- As the configuration changes, Terraform is able to determine what changed and create incremental execution plans which can be applied.

WORKING

- Let's think, we want to create AWS Instance, elastic ip and add the DNS entry for the instance.
- We just need to specify the “END state” that we want using TERRAFORM.
- Terraform will calculate the dependencies and create each of the resources in the correct order.
- Terraform also called as **CLOUD PROVISIONER**.

Example of Imperative

1. VM create with “t3.micro” instance type
2. Bring the VM down
3. Change the instance type to “t3.medium”
4. Bring the VM up

Example of Declarative

1. Tell terraform
2. Changing instance type from “t3.micro” ➞ “t3.medium”

Advantages of Terraform over Config Mgmt

- **Terraform maintains the State of the Infrastructure**
 - Meta data of the infrastructure
 - When it was created?
 - What was created?
 - What was updated?
 - What was partly deleted?

Advantages of Terraform over Config Mgmt

- **Terraform maintains the State of the Infrastructure**
- Re-Usable templates across all the Cloud Infra.
- Leverage TERRAFORM in the existing continuous pipeline
- Terraform can handle immutable method of server deployment.
- Since Terraform remembers the current state of the infra, making and applying changes is easy compared to provisioning tools (Chef, Puppet & Ansible).

Sample AWS Terraform script

```
provider "aws" {  
  region    = "us-west-2"  
  access_key = "accesskey"  
  secret_key = "secretkey"  
}  
  
resource "aws_instance" "example" {  
  ami = "ami-8803e0f0"  
  instance_type = "t2.micro"  
}
```

Commands to execute

File extension should be ☾ “example.tf”

```
[root@terraform-vm proj1]# pwd  
/root/proj1
```

terraform init ☾ this is to initiate the folder for terraform

terraform apply ☾ This would create /modify / delete the resources as per the “*.tf” file.

terraform destroy ☾ This would clean up all the resources that was created.



Thank you

